

CONECTEA

Documento Central Obrigatorio

Regras operacionais, DS V2, seguranca, modularidade e fechamento por escopo

Versao 4.2 - Fonte obrigatoria viva do Projeto ConeCTEA

Produto final em construcao - nao tratar como MVP.

Aplicacao: qualquer tela, fluxo, feature, service, RPC, migration, Edge Function, visual, dado ou integracao.

Regra central: auditar, classificar, modularizar, proteger, validar e so entao avancar.

Regra financeira: ConeCTEA deve operar com custo zero; recurso pago nao entra sem autorizacao e caixa definido.

Este documento substitui a versao 2.0 e descarta o rascunho v3.0 anterior.

Fixar como fonte do Projeto ConeCTEA e revisar em todo chat novo antes de prompts ou decisoes sensiveis.

Quando houver conflito, prevalece a regra mais segura, mais restritiva e o codigo real atual.

Indice de consulta rapida

Este documento centraliza as regras obrigatorias do ConeCTEA. Ele deve ser lido antes de qualquer prompt para Antigravity, Codex ou outro executor, e antes de qualquer decisao que envolva UI, DS V2, seguranca, dados, Supabase, Play Console, custos ou arquitetura.

1. Regra maxima e aplicacao por escopo
2. Produto, custo zero e limites financeiros
3. Papeis de Lucas, ChatGPT, Antigravity e Codex
4. Ordem obrigatoria dentro de cada escopo
5. Auditoria, relatorio e evidencias
6. Camada Padronizada V2 e DS V2 viva
7. Legado, lacuna e regra de migracao
8. Tres camadas visuais: local, compartilhada e global
9. Semantica visual, DsCores e DsTokenStatus
10. UI/UX neurodivergente e responsividade global
11. Arquitetura modular por features
12. Seguranca, LGPD, Play Console e dados sensiveis
13. Supabase, CRUD, RLS, GRANTS, RPCs e Edge Functions
14. Fluidez local e otimizacao de egress
15. Documentacao, validacao, commit e checklist final
16. GitHub Desktop: commit e push manuais
17. Governanca de atualizacao do documento central

Regra de leitura obrigatoria

Cada nova frente deve ser tratada como um novo escopo. Ao terminar um escopo, executar a auditoria final e o checklist. So depois iniciar o proximo escopo, repetindo todas as etapas. Nada neste documento e opcional.

1. Regra maxima e aplicacao por escopo

O ConeCTEA deve ser tratado como produto final em construçao. Ele pode evoluir por etapas, mas nenhuma etapa deve nascer como improvisado, MVP descartavel ou código temporario sem controle.

Este documento se aplica a qualquer tela, fluxo, feature, widget, helper, service, model, route, migration, RPC, Edge Function, integracao, documento, prompt, validacao, correcao visual, refino de responsividade, otimizacao, seguranca ou preparacao para Play Console.

Regra central

Dentro de cada escopo: entender, auditar, classificar, modularizar quando necessario, proteger dados, respeitar DS V2, validar responsividade, checar seguranca, otimizar fluidez, otimizar egress, relatar evidencias e so entao avancar.

1.1 Escopo e subescopo

Escopo e a frente autorizada por Lucas no chat atual. Pode ser uma tela, uma parte de uma tela, um fluxo de CPF, uma frente de notificacoes, uma revisao de admin, uma evolucao de DS V2, uma migration ou uma auditoria.

Subescopo e uma parte menor dentro da frente. Exemplo: dentro do fluxo de CPF, pode haver subescopo de visual, subescopo de RPC, subescopo de GRANT, subescopo de validacao local e subescopo de Play Console/LGPD.

Cada escopo e subescopo deve respeitar este documento. Fechar um escopo nao autoriza pular auditoria no proximo.

1.2 O que significa pronto

Uma frente nao esta pronta porque compila, porque a tela abre ou porque o executor escreveu "feito com sucesso". Pronto significa: escopo cumprido, nenhuma alteracao fora do pedido, dados protegidos, DS V2 respeitada, responsividade considerada, validacoes executadas, riscos classificados, relatorio factual e validacao manual/visual quando houver UI.

2. Produto, custo zero e limites financeiros

Regra financeira fundamental

O ConeCTEA deve operar com custo financeiro zero. O caixa do projeto e zero. Nenhuma solucao, API, servico, banco, envio de e-mail, notificacao, armazenamento, analytics, OCR, IA, mapa, SMS, hospedagem, automacao ou integracao pode depender de pagamento para funcionar.

Sempre que uma frente exigir recurso novo, a primeira decisao tecnica deve ser buscar uma opcao gratuita que atenda ao ConeCTEA. Se nao existir, a segunda decisao deve ser desenhar uma alternativa gratuita, reduzir escopo, mudar estrategia ou adiar a funcionalidade.

2.1 Proibicoes de custo

- Nao contratar servico pago, API paga, add-on pago, plano Pro, assinatura, pacote de credits, SMS pago, e-mail pago, OCR pago, IA paga, storage pago ou automacao paga sem autorizacao explicita de Lucas e existencia real de caixa.
- Nao criar dependencia que funcione apenas em trial, limite promocional ou cota temporaria.
- Nao implementar funcionalidade que pare de funcionar quando o limite gratuito acabar sem plano de contingencia.
- Nao resolver problema gratuito atual empurrando para plataforma paga.
- Nao esconder custo indireto: egress, storage, invocacao, imagem, log, build, webhook, fila, SMTP, push, analytics ou retencao tambem contam.

2.2 Checklist de custo antes de recurso novo

- ☐ O recurso e gratuito hoje?
- ☐ O plano gratuito cobre o uso esperado do ConeCTEA?
- ☐ Existe custo por envio, chamada, usuario, arquivo, egress, storage ou invocacao?
- ☐ Existe alternativa gratuita por Google Apps Script, Supabase Free, OneSignal Free, recurso nativo ou fluxo manual?
- ☐ A funcionalidade pode ser redesenhada para custo zero?
- ☐ O relatorio informou risco de custo futuro ou limite de cota?
- ☐ Lucas autorizou explicitamente qualquer execucao financeira?

Se uma unica resposta indicar custo ou risco financeiro nao resolvido, a frente deve parar para decisao de produto. A regra e criar o ConeCTEA de forma sustentavel dentro do custo zero.

3. Papeis de Lucas, ChatGPT, Antigravity e Codex

3.1 Lucas

Lucas decide produto, prioridade, escopo, visual final, aceite manual e autorizacao de commit/push. Toda mudanca sensivel exige autorizacao clara de Lucas.

3.2 ChatGPT

ChatGPT organiza o raciocinio, identifica riscos, define escopo, gera prompts quando Lucas pedir, audita relatorios, confere coerencia com este documento e impede que tarefas avancem sem evidencia. ChatGPT nao deve sugerir implementacao direta em area sensivel sem auditar e mapear impacto.

3.3 Antigravity

Antigravity e executor limitado. Ele executa apenas o escopo pedido. Nao decide arquitetura, produto, seguranca, Supabase, DS V2 ou refatoracao ampla. Problemas fora do escopo devem ser relatados, nao corrigidos automaticamente.

3.4 Codex

Codex pode fazer auditoria local, inventario, leitura de repo, cruzamento de chamadas, busca de legados e analise de uso real. Quando o pedido for auditoria, Codex deve permanecer em somente leitura. Implementacao por Codex tambem exige prompt fechado, autorizacao e relatorio factual.

Fluxo obrigatorio

Lucas explica -> ChatGPT organiza -> executor audita/implementa so o escopo -> executor relata -> Lucas cola relatorio -> ChatGPT audita -> Lucas valida visualmente quando houver UI -> commit/push apenas com autorizacao explicita.

4. Ordem obrigatoria dentro de cada escopo

A ordem abaixo e obrigatoria. Ela vale para CPF, admin, carteirinha, home, auth, documentos, notificacoes, suporte, DS V2, Supabase, Play Console ou qualquer outra frente.

Etapa	Obrigacao
0. Custo zero	Verificar se qualquer recurso novo tem custo financeiro, limite gratuito, egress ou dependencia paga.
1. Escopo	Definir exatamente o que entra e o que fica fora.
2. Auditoria somente leitura	Mapear arquivos, tela, fluxo, dados, DS V2, legado, services, Supabase e riscos sem alterar nada.
3. Classificacao modular	Decidir se o trecho e local, compartilhado controlado, global DS V2, helper, service ou feature.
4. DS V2 e legado	Usar o que existe na DS V2. Se faltar e for reaproveitavel, evoluir a DS V2 antes de usar legado.
5. UI/UX e responsividade	Checar clareza, semantica, contraste, 360dp a 480dp, SafeArea, teclado e Android-first.
6. Seguranca/LGPD/Play Console	Mapear dados pessoais/sensíveis, logs, documentos, privacidade e declaracoes reais.
7. Supabase/CRUD/GRANTS	Checar create/read/update/delete, RLS, policies, grants, RPCs, Edge Functions e permissao.
8. Implementacao em microfrente	Alterar apenas arquivos autorizados, com menor recorte seguro e reversivel.

9. Fluidez local	Revisar rebuilds, setState, streams, dispose, loading, scroll, cache local e performance de uso.
10. Egress	Reduzir chamadas desnecessarias, select generico, stream pesada, leituras repetidas e payload excessivo.
11. Validacao	Executar validacoes proporcionais: analyze, build quando aplicavel, diff, status, teste/manual.
12. Relatorio e aceite	Relatar evidencias, riscos, pendencias, status Git, validacao visual e recomendacao de commit.

5. Auditoria, relatorio e evidencias

Auditoria somente leitura significa nao alterar nada. Se qualquer arquivo novo, modificado ou removido aparecer no status final sem estar no status inicial, a auditoria deixou de ser somente leitura.

5.1 Auditoria obrigatoria antes de implementar

- Ler codigo real; documentacao antiga pode estar desatualizada.
- Identificar arquivos da tela/fluxo, widgets, helpers, services, models, routes e dependencias.
- Mapear DS V2 usada, lacunas da DS V2, legados ativos, hardcoded visual e duplicacoes.
- Mapear dados pessoais, dados sensiveis, Supabase, Auth, Storage, RLS, GRANTS, RPCs, Edge Functions, Drive/GAS, OneSignal e logs.
- Separar fato, inferencia, risco, pendencia futura e recomendacao.
- Informar git status inicial e final.

5.2 Relatorio bom

Relatorio bom e evidencia. Deve trazer objetivo, status Git inicial, comandos, arquivos lidos, arquivos alterados/criados/removidos, resumo do diff, validacoes, riscos, limitacoes, status Git final e confirmacao de que nao houve stage, commit ou push quando nao autorizado.

5.3 Relatorio ruim

Relatorio ruim diz apenas "feito", "ajustado" ou "sucesso" sem evidencias. Se faltar status Git, arquivos lidos, arquivos alterados, validacoes, riscos ou limitacoes, o trabalho nao deve ser aceito sem complemento.

Regra de aceite

ChatGPT nao deve aceitar relatorio generico. Lucas nao deve autorizar commit se o relatorio nao prova escopo, seguranca, validacao e estado do Git.

6. Camada Padronizada V2 e DS V2 viva

A DS V2 e viva, expansiva e e a base de criacao do ConeCTEA. Ela nao e apenas uma pasta visual: ela representa a camada padronizada reutilizavel do projeto. Isso inclui tokens, componentes, padroes visuais, campos, helpers globais e servicos utilitarios que nasceram para evitar repeticao e manter consistencia.

6.1 DS V2 oficial

- lib/core/design_system_v2/ e o nucleo oficial.
- Tudo que nasce como componente, token ou padrao visual global deve preferencialmente usar prefixo Ds e morar dentro da DS V2.

- design_system_v2.dart e o barrel oficial. Novas telas devem preferir o barrel quando possivel.
- Antes de usar qualquer item da DS V2, conferir a API real no codigo atual. Nao inventar parametros, variantes ou nomes.

6.2 DS V2 oficial expandida

Algumas pastas nasceram com a padronizacao V2, mesmo que historicamente nao tenham seguido o prefixo Ds. Elas fazem parte integral do ecossistema V2 e devem ser tratadas como padroes oficiais, nao como legado.

- lib/core/campos_cadastrais/ - parte integral da DS V2. Campos como CPF, e-mail, telefone, estado, cidade e demais campos padronizados devem ser usados nos fluxos correspondentes.
- lib/core/servicos/localizacao/ - parte do ecossistema V2 de padronizacao global de localizacao. Nao e visual, mas e padrao reutilizavel do projeto.
- Helpers globais futuros devem nascer em local claro, com nome claro e rastreabilidade de versao/padrao, para que uma futura V3 consiga identificar o que nasceu na V2.

6.3 Pergunta obrigatoria antes de criar algo

- ☐ O que estou fazendo agora ja existe na DS V2?
- ☐ Existe parcialmente na DS V2? Se sim, devo completar/evoluir a DS V2 antes de criar solucao solta?
- ☐ Isso pode ser reutilizado em outra tela, fluxo ou feature?
- ☐ Isso deve ser local, compartilhado controlado ou global?
- ☐ Estou usando legado apenas porque e mais rapido?
- ☐ Estou criando uma coisa solta que futuramente vai obrigar manutencao manual tela por tela?

Se algo puder ser reaproveitado no projeto como um todo, a preferencia e evoluir a DS V2 ou a camada V2 correspondente. E melhor expandir a DS V2 do que deixar solucao solta.

7. Legado, lacuna e regra de migracao

7.1 Legado nao e referencia de futuro

Legado tolerado e aquilo que ainda precisa existir porque telas antigas dependem dele. Ele fica ativo ate a migracao dessas telas, mas nao pode ser reutilizado em nova tela, refatoracao nova ou componente novo.

7.2 Legado proibido para novos usos

- Tudo que comeca com Premium em camada visual ou padronizavel e legado proibido para novo uso.
- Tudo visual ou padronizavel que comeca com Conectea e legado proibido para novo uso, salvo excecao explicitamente reclassificada por auditoria de Lucas.
- AppColors, AppTextStyles, AppSpacing, AppRadius, AppShadows, StatusVisualTokens, ConecteaVisualTokens e caminhos paralelos de cor/estilo nao devem ser usados em novas telas.
- PremiumAuthBackground, AppBackground, LoadingShimmer e equivalentes antigos nao devem ser usados em novas frentes; se a frente precisar disso, criar/evoluir a DS V2 primeiro.
- InputDecoration, TextStyle, Color(0x...), LinearGradient, BoxShadow e ButtonStyle hardcoded em tela nova so podem existir quando for caso local auditado e justificado. O padrao e usar DS V2.

7.3 Regra para lacuna DS V2

Faltou na DS V2?

Se uma frente precisa de background, loading, skeleton, erro, empty state, auth background, scaffold, upload, bottom sheet ou outro padrao ainda inexistente, a primeira tarefa e evoluir a DS V2. Nao reutilizar legado para ganhar velocidade.

Exemplo: se uma tela precisa de background, nao usar AppBackground ou PremiumAuthBackground em nova frente. Criar DsBackground, DsScaffold, DsAuthBackground ou o nome tecnicamente adequado dentro da DS V2, usar no escopo atual e migrar telas antigas quando elas entrarem em escopo.

7.4 Legado morto

Legado morto e arquivo orfao ou classe sem uso real. Ele nao deve ficar no projeto por apego historico. Deve ser removido em microfrente segura, com auditoria de uso, git status, diff e validacao.

8. Tres camadas visuais: local, compartilhada e global

A camada visual do ConeCTEA deve ser classificada antes de criar, mover ou promover widgets. A regra evita dois erros: transformar a DS V2 em deposito de qualquer widget especifico, ou duplicar componentes iguais em varias telas.

8.1 Camada local de feature/tela

Widget local pertence a uma tela ou feature especifica. Ele pode e deve usar DS V2 por dentro, mas nao vira DS V2 automaticamente.

- Usado em uma tela ou em uma composicao muito especifica.
- Depende de regra, dado, model, status ou contexto de uma feature.
- Pode ficar em features/.../widgets ou perto da tela.
- Exemplo: composicoes especificas da carteirinha digital, como frente/verso/background visual da carteirinha.

8.2 Camada compartilhada controlada

Compartilhado controlado e a camada do meio. E usado por duas ou tres telas do mesmo dominio, exatamente da mesma forma. Nao e "parecido": e igual.

- Aparece em duas ou tres telas, normalmente do mesmo dominio.
- Tem o mesmo visual, os mesmos dados, o mesmo comportamento, a mesma regra de loading/erro e a mesma fonte/helper de dados.
- Se uma tela muda dados, layout, estado, fonte de consulta ou comportamento, deixa de ser compartilhado e volta a ser local.
- A preferencia e ficar dentro da feature dona do conceito, por exemplo features/carteirinhas/shared/, evitando um shared global virar deposito.

8.3 Camada global / DS V2

Global e aquilo que virou linguagem do app inteiro ou estrutura reutilizavel ampla. Pode aparecer em varias features com variacoes controladas por API pequena.

- Usado em muitas telas ou features, ou claramente esperado como padrao de produto.
- Nao depende de regra especifica de uma feature.
- Pode ser componente, token, padrao, helper global ou servico utilitario padronizado.

- Existem globais identicos e globais estruturais. DsCard e DsBotao sao estruturais: a base e igual, o conteudo muda.

Regra de promocao

Uma tela = local. Duas ou tres telas com uso exatamente igual = compartilhado controlado. Varias telas/features ou linguagem do app = global/DS V2. Se ganhou variacoes demais, reavaliar: DS parametrizada ou widgets locais separados.

9. Semantica visual, DsCores e DsTokenStatus

No ConeCTEA, cor e comunicacao, mas nunca comunica sozinha. A intencao visual nasce da combinacao entre cor, icone, texto, forma, posicao, contexto, componente e consequencia da acao.

9.1 Caminho unico de cores

- DsCores e o caminho unico para cores estruturais e intencoes visuais.
- DsCorVisual representa area, acao ou contexto visual.
- DsTokenStatus e separado e deve ser usado somente para status de carteirinha ou solicitacao.
- Nao recriar DsTokensVisuais, DsTokenVisual, ConecteaVisualTokens ou qualquer caminho paralelo de cores semanticas.
- Status persistido e uma coisa; intencao visual e outra.

9.2 Regra de composicao visual

Composicao obrigatoria

Intencao visual = cor + icone + texto + contexto + consequencia da acao. Cor sozinha nao basta.

- Dados protegidos: texto claro, icone coerente, token de dados protegidos/privacidade e explicacao de analise.
- Solicitar correcao: intencao de revisao/correcao, geralmente ambar/laranja controlado, sem parecer erro destrutivo.
- Excluir/remover: intencao de perigo, vermelho controlado, texto explicito e confirmacao.
- Carteirinha: area/produto usa intencao visual; status formal usa DsTokenStatus.

9.3 Regras cromaticas essenciais

- Fundo principal deve permanecer Night Blue / Dark Glass Premium.
- Cards devem ser neutros; cor semantica aparece em detalhe: icone, borda, selo, faixa, glow sutil ou CTA.
- Nao pintar card inteiro com cor do modulo sem justificativa.
- Contraste vem antes de estetica.
- Evitar muitas cores fortes, excesso de glow, verde para qualquer acao e vermelho para pendencia simples.

10. UI/UX neurodivergente e responsividade global

A interface do ConeCTEA deve funcionar para pessoas autistas, pessoas com TDAH, outras neurodivergencias, familiares, responsaveis, administradores e usuarios com baixa familiaridade digital. O objetivo nao e estetica "para neurodivergentes", mas experiencia clara, previsivel, confortavel e segura.

10.1 Principios de UI/UX

- Clareza antes de ornamentacao.
- Previsibilidade antes de surpresa.
- Conforto antes de densidade.
- Legibilidade antes de impacto visual.
- Seguranca emocional antes de efeito bonito.
- Acao clara antes de muitas opcoes.
- Linguagem simples antes de texto tecnico.
- Consistencia antes de criatividade solta.

10.2 Responsividade Android-first

- Chrome e fallback visual inicial, nao validacao principal.
- Pensar em Android real, principalmente 360dp a 480dp.
- 360dp e o teste mental minimo; em caso de duvida, priorizar 360dp.
- Considerar SafeArea, status bar, navigation bar, 3 botoes, gestos, teclado aberto, notch, cantos arredondados, fonte ampliada e aparelhos simples.
- Nao resolver tela estreita encolhendo tudo; reorganizar, quebrar em blocos, permitir scroll seguro ou mover detalhes para tela/modal.

10.3 Proibicoes visuais recorrentes

- Texto pequeno para informacao essencial.
- Ellipsis em motivo de reprova, prazo, erro, pendencia, instrucao ou dado necessario para agir.
- Botao esmagado, texto longo em selo, Row sem Flexible/Expanded, altura fixa perigosa.
- MediaQuery usado para calcular largura global quando o espaco real e local.
- Grid em tela estreita quando lista vertical seria mais legivel.
- Brilho ou animacao competindo com informacao sensivel.

11. Arquitetura modular por features

O ConeCTEA usa arquitetura modular por features. Modularidade e obrigatoria para evitar telas gigantes, arquivos com milhares de linhas, mistura de visual com regra sensivel e manutencao impossivel.

11.1 Antes de crescer uma tela

- ☐ O trecho pertence a tela, a feature, a camada compartilhada, a DS V2, a service, ao model ou a uma migration?
- ☐ O arquivo esta ficando grande demais para auditar?
- ☐ Existe bloco visual repetido ou regra local que merece extracao?
- ☐ A extracao reduz risco ou cria complexidade desnecessaria?
- ☐ Existe dado sensivel, Supabase, Auth, permissao ou documento misturado com UI?
- ☐ Daria para modularizar agora em microrecorte seguro?

11.2 Modularizar cedo, mas sem refatorar tudo

Se um trecho ja esta grande, repetido ou atrapalhando leitura, modularizar em microfrente segura. Nao esperar o arquivo virar 3 mil linhas para modularizar. Ao mesmo tempo, modularidade nao autoriza reescrever a tela inteira sem escopo.

- Tela coordena fluxo, estado local e composicao principal; nao deve guardar tudo.
- Widget de feature guarda composicao especifica e pode usar DS V2 por dentro.
- Helper de feature guarda formatacao/calc local leve; nao deve esconder regra critica.
- Service guarda integracao/regra de acesso; se envolver dados sensiveis, auditar melhor.
- DS V2 guarda padrao global reutilizavel, sem regra de negocio sensivel.

12. Seguranca, LGPD, Play Console e dados sensiveis

Seguranca nao e etapa final. Ela orienta desenho, texto, fluxo, armazenamento, logs, permissao, documentacao e publicacao. Se envolver dado pessoal ou sensivel, desacelerar e auditar.

12.1 Dados de cuidado obrigatorio

- CPF, e-mail, telefone, data de nascimento, cidade/estado e dados de conta.
- Dependentes, criancas, adolescentes, responsaveis, vinculo familiar e contato de emergencia.
- Documentos com foto, laudos, CID, tipo sanguineo, raca/cor e informacoes relacionadas a neurodivergencia/deficiencia.
- QR Code, TEA ID, identificadores internos, status, motivos de pendencia, recusa, suspensao e dados administrativos.
- Permissoes, cargos, payloads, fileId, URLs de Drive, tokens, chaves e logs.

12.2 Regras de logs e exposicao

- Nao logar CPF completo, e-mail completo sem necessidade, documento, laudo, CID, fileId, URL completa, QR bruto, token, chave, payload sensivel ou dado administrativo desnecessario.
- Mascarar dados em tela quando possivel.
- Nao colocar CPF em URL, nome de arquivo exposto, notificacao, analytics ou logs.
- Nao prometer seguranca, exclusao, criptografia, anonimizacao ou descarte se o codigo real nao prova isso.
- Politica de Privacidade, Termos, Data Safety e app real precisam contar a mesma verdade.

12.3 Play Console

Políticas do Google Play podem mudar. Sempre que a frente envolver publicacao, Data Safety, Health Apps, politica de privacidade, declaracao de saude, permissao Android ou resposta ao Play Console, consultar fonte oficial atual e auditar o codigo real antes de responder.

13. Supabase, CRUD, RLS, GRANTs, RPCs e Edge Functions

Supabase e nucleo sensivel do ConeCTEA. RLS, GRANTs, RPCs, Edge Functions, Storage, Auth, roles, realtime, validade de carteirinha e prazos devem ser tratados como area critica.

13.1 Antes de tocar banco ou service

- ☐ Qual tabela, view, RPC, trigger, Edge Function, policy, GRANT ou service participa do fluxo?
- ☐ Quem pode criar, ler, atualizar e apagar cada dado?
- ☐ Existe dado pessoal/sensivel?
- ☐ Existe risco de enumeracao, vazamento, update amplo, N+1, stream pesada ou permissao excessiva?
- ☐ O client realmente precisa desses campos?
- ☐ A regra depende do servidor/banco e do fuso America/Sao_Paulo?

- ☐ Ha migration reversivel e pequena?
- ☐ Lucas autorizou explicitamente?

13.2 Regras de RPC, RLS e GRANTS

- RLS/GRANT/policy nao devem ser alterados para silenciar alerta sem entender desenho real.
- Tabelas privadas com RLS ligado e sem policy podem estar corretas se o acesso for mediado por RPC segura.
- Funcao SECURITY DEFINER exige justificativa, search_path explicito, auth.uid()/ownership quando aplicavel e grants restritos.
- Funcoes trigger-only nao devem ficar executaveis diretamente por anon/authenticated/PUBLIC quando isso nao for necessario.
- RPC chamada pelo app nao pode ter grant revogado sem validar fluxo completo.
- Evitar select(*) e payload amplo; consultar apenas campos necessarios.

13.3 Divida tecnica acoplada ao escopo

Se a frente atual mexe em CPF e existe divida em RPC de CPF, validacao de CPF, GRANT de CPF, enumeração por CPF ou log de CPF, essa divida entra no escopo apos auditoria e autorizacao. Se a divida esta em OneSignal ou pg_net e nao afeta CPF, ela e registrada para frente futura.

14. Fluidez local e otimizacao de egress

Otimização de tela e otimização de egress sao coisas diferentes. As duas sao obrigatorias no fechamento de escopo, mas devem ser analisadas separadamente.

14.1 Fluidez local

- Foco na experiencia da pessoa usando o app.
- Revisar rebuilds desnecessarios, setState amplo, StreamSubscription, dispose, controllers, listeners, memory leak, loading/empty/error states, scroll e responsividade.
- Evitar streams no build e listeners sem cancelamento no dispose.
- Garantir feedback previsivel, loading que nao pula layout e tela que continua usavel em aparelho simples.

14.2 Egress e consumo de banco

- Foco em nao chamar banco toda hora sem necessidade e nao gastar egress inutil.
- Evitar select(*) quando a tela usa poucos campos.
- Evitar streams pesadas, leituras repetidas, historico infinito, payload grande e N+1 no client.
- Usar limit, paginacao, cache local, RPC leve ou consulta agregada quando fizer sentido e for seguro.
- Otimizar egress sem esconder dado necessario nem quebrar seguranca.

Regra pratica

Fluidez local responde: a tela e confortavel, rapida e previsivel para a pessoa? Egress responde: o banco esta sendo chamado apenas quando precisa, com o menor payload seguro?

15. Documentacao, validacao, commit e checklist final

15.1 Documentacao

A documentacao deve acompanhar o projeto, mas o codigo real prevalece quando houver conflito. Documentar nao deve virar desculpa para alterar README, docs, scripts ou arquivos de referencia sem autorizacao. Ao final de frentes maiores, registrar decisoes tecnicas, riscos, padroes novos e migracoes realizadas.

15.2 Validacoes tecnicas

- Microfrentes pequenas podem ter validacao proporcional, mas fechamento de tela/fase exige validacao completa.
- Alteracao Dart/Flutter: flutter analyze como piso minimo.
- Fechamento relevante: flutter build apk --debug quando aplicavel.
- Sempre que houver alteracao: git diff --check, git diff --name-only, git diff --stat e git status --short no relatorio.
- Build limpo nao prova UI boa. Analyze limpo nao prova seguranca. Validacao tecnica e piso, nao aprovacao final.

15.3 Commit e push

No ConeCTEA, commit e push sao atos manuais de Lucas pelo GitHub Desktop. ChatGPT, Antigravity, Codex ou qualquer outro executor nao devem executar commit, push ou staging por conta propria.

- Commit so depois de escopo conferido, relatorio auditado, validacoes limpas e autorizacao explicita de Lucas.
- UI exige validacao visual/manual de Lucas antes de commit quando a frente for visual.
- Lucas faz staging, commit e push manualmente pelo GitHub Desktop.
- Antigravity e Codex nao podem fazer staging, commit ou push.
- ChatGPT nao deve pedir git add ., git add -A, git commit ou git push como caminho padrao do projeto.
- Commit autorizado nao significa push autorizado. Push tambem e decisao manual de Lucas no GitHub Desktop.
- Se git status estiver sujo por arquivos nao explicados, nao commitar.
- Depois de sugerir mensagem de commit, ChatGPT deve aguardar Lucas confirmar que comitou. Sem confirmacao, o estado correto e commit sugerido, mas nao confirmado.

15.4 Titulo e descricao de commit para GitHub Desktop

Quando uma frente estiver aprovada para commit, ChatGPT deve fornecer a Lucas o titulo e a descricao do commit prontos para copiar no GitHub Desktop. A mensagem deve ser curta, rastreavel, segura e sem dados sensiveis.

Formato obrigatorio

ChatGPT deve entregar: Titulo do commit, Descricao do commit e Observacao para Lucas revisar os arquivos no GitHub Desktop antes de confirmar commit e push.

- Titulo: curto, claro e rastreavel. Evitar genericos como ajustes, melhorias, update ou varias coisas.
- Descricao: explicar o escopo, o que mudou, areas/arquivos principais, validacoes executadas e pendencias conhecidas.
- Nao incluir CPF, e-mail, nome de pessoa, protocolo real, ID interno, fileld, URL completa, QR bruto, token, chave, payload ou dado sensivel em titulo ou descricao.

- Preferir prefixos como docs, ds, ui, fix, security, supabase, auth, account, cpf, admin, perf, egress, refactor ou chore.
- Se houver pendencia visual, risco residual ou validacao nao executada, isso deve aparecer na descricao antes de Lucas decidir commitar.

15.5 Checklist final obrigatorio

- ☐ Escopo atual foi respeitado?
- ☐ Custo zero foi respeitado?
- ☐ Auditoria inicial foi feita?
- ☐ DS V2 foi usada/evoluida quando necessario?
- ☐ Legado nao foi reutilizado indevidamente?
- ☐ Widgets foram classificados como local, compartilhado controlado ou global?
- ☐ Modularidade foi aplicada sem refatoracao ampla?
- ☐ UI/UX neurodivergente e semantica visual foram consideradas?
- ☐ Responsividade 360dp a 480dp foi considerada?
- ☐ Dados pessoais/sensíveis foram mapeados e protegidos?
- ☐ Supabase, CRUD, RLS, GRANTS, RPCs e logs foram avaliados quando aplicavel?
- ☐ Fluidez local foi revisada?
- ☐ Egress foi revisado?
- ☐ Validacoes tecnicas foram executadas ou justificadas?
- ☐ Relatorio trouxe evidencias reais?
- ☐ Lucas validou visualmente quando havia UI?
- ☐ Nao houve staging, commit ou push pelo executor?
- ☐ ChatGPT forneceu titulo e descricao de commit apenas se a frente estiver pronta para Lucas decidir?
- ☐ Pendencias futuras foram registradas sem invadir escopo?

Fechamento

Se qualquer item obrigatorio nao passou, a frente nao esta fechada. Corrigir, auditar novamente ou registrar pendencia conforme o risco. So iniciar o proximo escopo depois do fechamento consciente do atual.

16. GitHub Desktop: commit e push manuais

Esta secao integra definitivamente a regra criada na v4.1 ao corpo principal do documento. Ela deve ser lida antes de qualquer fechamento de frente.

Regra principal

Lucas e a unica pessoa que realiza staging, commit e push no fluxo normal do ConeCTEA, usando o GitHub Desktop. ChatGPT fornece mensagem segura; Antigravity e Codex apenas relatam evidencias.

16.1 O que cada papel faz

- Lucas revisa os arquivos no GitHub Desktop, escolhe o que entra no commit, escreve/cola titulo e descricao, confirma o commit e decide se fara push.
- ChatGPT audita o relatorio, identifica se a frente pode ser commitada e fornece titulo/descricao seguros quando fizer sentido.

- Antigravity/Codex nao executam commit, push, staging, git add, git commit, git push ou comandos equivalentes sem autorizacao explicita e excepcional de Lucas.
- Relatorio de executor deve informar claramente se nao houve staging, commit ou push.

16.2 Modelo de mensagem de commit

Modelo

Titulo do commit: <tipo>: <resumo curto do escopo> | Descricao do commit: o que mudou, areas/arquivos principais sem expor dado sensivel, validacoes executadas e pendencias conhecidas | Observacao para Lucas: revisar arquivos no GitHub Desktop antes do commit e fazer push manualmente apenas se esse for o proximo passo autorizado.

16.3 Regra de nao assumir commit

Depois de fornecer titulo e descricao, ChatGPT deve aguardar Lucas informar que realizou o commit pelo GitHub Desktop. Enquanto Lucas nao confirmar, o estado correto e: commit sugerido, mas nao confirmado. Push tambem deve ser confirmado por Lucas quando for relevante ao proximo passo.

17. Governanca de atualizacao do documento central

O Documento Central Obrigatorio e uma fonte viva e versionada do Projeto ConeCTEA. Ele pode e deve ser atualizado quando novas regras permanentes forem decididas durante o percurso, especialmente apos escopos grandes, auditorias relevantes, evolucoes da DS V2, mudancas de seguranca, mudancas de arquitetura, novas exigencias do Play Console ou decisoes de produto.

Regra de preservacao

Nada que ja foi definido como regra obrigatoria deve ser apagado ou enfraquecido sem registro claro de substituicao. A forma correta de evoluir e adicionar regra nova, registrar versao nova e indicar quando uma regra nova complementa, substitui ou restringe uma regra anterior.

17.1 Quando atualizar este documento

- Depois de escopo grande que criou regra permanente de produto, DS V2, seguranca, fluxo, modularidade ou validacao.
- Quando a DS V2 ganhar novo componente, token, padrao, helper global ou regra semantica que deve ser lembrada em novos chats.
- Quando uma lacuna deixar de ser lacuna porque virou DS V2 oficial.
- Quando uma regra de legado mudar de tolerado para morto/removivel ou de tolerado para proibido.
- Quando um novo risco de custo, egress, Play Console, LGPD, Supabase, Git ou arquitetura for descoberto.
- Quando uma decisao de Lucas precisar virar regra permanente para todos os proximos escopos.

17.2 Como atualizar

- ☐ Criar nova versao numerada do PDF, mantendo historico.
- ☐ Informar o que foi adicionado, o que foi refinado e por que.
- ☐ Nao esconder regra nova em local dificil quando ela muda fluxo central; integrar no corpo principal na proxima versao completa.
- ☐ Manter o documento mais novo como fonte fixada principal do Projeto.
- ☐ Se houver conflito entre versoes, prevalece a versao mais nova quando ela for mais segura, mais restritiva, mais gratuita, mais acessivel ou mais alinhada ao produto final.
- ☐ Atualizacao de documento nao autoriza alteracao de codigo.Codigo so muda por escopo proprio, com auditoria e prompt autorizado.

17.3 O que pode ser atualizado

- Inventario da DS V2, incluindo componentes, tokens, padroes, campos cadastrais, localizacao e helpers padronizados.
- Regras de semantica, semiotica, DsCores, DsCorVisual e DsTokenStatus.
- Classificacao de legado tolerado, legado proibido e legado morto.
- Regras de modularidade local, compartilhada controlada e global.
- Regras de responsividade, Android-first, validacao visual e acessibilidade.
- Regras de seguranca, dados sensiveis, Play Console, Supabase, CRUD, RLS, GRANTs, RPCs, Edge Functions e egress.
- Regras de custo zero e alternativas gratuitas.
- Regras de GitHub Desktop, titulo/descricao de commit e confirmacao manual de push.

17.4 Regra final da governanca

Ao final de uma frente importante, ChatGPT deve perguntar se alguma decisao tomada virou regra permanente. Se sim, a proxima acao correta e atualizar este Documento Central Obrigatorio em nova versao, sem apagar o historico e sem enfraquecer regras ja estabelecidas.

18. Frases de decisao para novos chats

Ao iniciar um novo chat ou nova frente, ChatGPT deve aplicar estas perguntas antes de gerar qualquer prompt:

- ☐ Qual e o escopo exato autorizado por Lucas?
- ☐ Essa frente toca dado sensivel, Supabase, Auth, RLS, migration, Play Console ou custo externo?
- ☐ O que existe hoje no codigo real?
- ☐ O que a DS V2 ja cobre?
- ☐ Existe legado no caminho? Ele e tolerado, proibido, morto ou lacuna?
- ☐ O trecho deve ser local, compartilhado controlado ou global?
- ☐ Ha divida tecnica acoplada ao escopo que deve ser resolvida agora?
- ☐ A solucao continua 100% gratuita?
- ☐ A tela funciona em Android 360dp a 480dp?
- ☐ O banco esta sendo usado com o menor payload seguro?
- ☐ Qual e o menor prompt seguro para o executor?

19. Resumo executivo obrigatório

Tema	Regra que nao pode ser esquecida
Custo	ConeCTEA deve funcionar com custo zero. Recurso pago nao entra.
Escopo	Cada frente cumpre todas as etapas antes da proxima.
Auditoria	Antes de implementar, auditar codigo real em somente leitura.
DS V2	Base viva e expansiva. Faltou padrao reutilizavel? Evoluir DS V2 primeiro.
Legado	Tolerado so para manter tela antiga funcionando; proibido para novo uso.
Modularidade	Nao deixar arquivo virar gigante; modularizar em microfrente segura.
Compartilhado	So e compartilhado se for exatamente igual em 2 ou 3 telas.
Global	Virou linguagem do app ou muitas telas: DS V2/global.
Seguranca	Dados pessoais/sensíveis exigem auditoria, minimizacao, logs seguros e verdade documental.
Supabase	RLS, GRANTs, RPCs, Edge e Auth exigem autorizacao e plano reversivel.
UI	Clareza, previsibilidade, contraste, baixa carga cognitiva e Android-first.
Fluidez	Experiencia local: rebuild, setState, streams, dispose, loading, scroll.
Egress	Banco: menos chamadas, menos payload, menos stream pesada, sem select(*) desnecessario.
GitHub Desktop	Lucas faz staging, commit e push manualmente. ChatGPT fornece titulo e descricao seguros.
Documento vivo	Novas regras permanentes entram em nova versao. Nao enfraquecer regra antiga sem registro claro.
Fechamento	Relatorio factual, validacoes, status Git, validacao visual quando UI, commit so autorizado.

20. Fontes consolidadas

Este documento central consolida os manuais internos, estudos tecnicos e decisoes de produto do ConeCTEA, incluindo operacao Lucas + ChatGPT + Antigravity, auditoria e relatorios, evolucao da DS V2, arquitetura modular por features, UI/UX neurodivergente, responsividade global, seguranca/LGPD/Play Console, DsCores, semantica visual, semiotica e psicologia das cores.

Quando houver diferenca entre documentacao, memoria de conversa e codigo, o codigo real atual prevalece. Quando houver conflito entre regras, prevalece a opcao mais segura, mais restritiva, mais gratuita, mais acessivel e mais alinhada ao produto final.